

Relatório de Atividades: Customização de Kernel e SO Minimalista

Disciplina: Sistemas Operacionais (2026/1)

Professor: Jeremias Moreira Gomes

Aluno: Fábio (Matrícula: 2321055)

1. O Desafio

O objetivo aqui foi colocar a mão na massa e construir um sistema Linux enxuto, configurado para rodar inteirinho na memória RAM (via *initramfs*) e emulado pelo QEMU. Foi uma ótima oportunidade para ver na prática como as *System Calls*, o *User Space* e o *Kernel* conversam.

2. Setup do Ambiente

Para garantir que tudo compilasse corretamente, usamos o padrão homologado no Laboratório Dell do IDP:

- **Hospedeiro:** Windows 11 com WSL2.
- **Distribuição:** Ubuntu 24.04 LTS.
- **Arquitetura Alvo:** x86-64.
- **Espaço em disco:** Recomendo pelo menos 10GB livres para lidar com a árvore do Kernel.

3. Passo a Passo

3.1. Preparando o terreno

Primeiro, atualizamos as ferramentas essenciais de compilação:

Shell

```
sudo apt-get update
sudo apt-get install -y build-essential libncurses-dev bison flex
libssl-dev libelf-dev pahole cpio qemu-system qemu-utils
qemu-system-x86
```

3.2. BusyBox (v1.36.1): A base do sistema

O BusyBox é essencial para termos os comandos básicos do UNIX em um binário único.

1. **Checkout:** git checkout 1_36_1
2. **Configuração (*make menuconfig*):**
 - **Networking Utilities -> tc:** Desabilitado (incompatível com kernels atuais).
 - **Settings -> Build Options -> Build static binary:** Habilitado (isso garante que rode na RAM sem depender de libs dinâmicas).
3. **Compilação:** Usei *make -j2* para não sobrecarregar o WSL2 e rodei *make install*. Isso gera a pasta *_install/* com tudo que precisamos.

3.3. Kernel Linux (v6.6.137)

1. **1. Checkout:** git checkout v6.6.137
2. **2. Configuração (*make menuconfig*):**
 - Ativei suporte a *initramfs* em *General setup*.
 - Garanti suporte a *64-bit*.
 - Em *Device Drivers*, habilitei o *devtmpfs* para montar o */dev* automaticamente.
3. **3. Erros de Assinatura:** Para evitar que a compilação travasse pedindo certificados, desabilitamos as checagens:

Shell

```
scripts/config --disable SYSTEM_TRUSTED_KEYS
scripts/config --disable SYSTEM_REVOCATION_KEYS
sed -i
's/CONFIG_SYSTEM_TRUSTED_KEYS="y"/CONFIG_SYSTEM_TRUSTED_KEYS=""/g
' .config
sed -i
's/CONFIG_SYSTEM_REVOCATION_KEYS="y"/CONFIG_SYSTEM_REVOCATION_KEY
S=""/g' .config
```

3.4. O Patch de Matrícula

Aplicamos o *matricula.patch* que injeta código na função *do_sysinfo* (kernel/sys.c). O código usa Assembly inline para pegar a versão do sistema, processar os dados e imprimir o token via *printk*.

Shell

```
git apply ../matricula.patch
```

Verificado com *git status* e pronto para compilar a *bzImage*.

3.5. Montagem do initramfs

Criamos a estrutura de diretórios e o script *init* que manda o sistema subir:

Shell

```
mkdir -p bin sbin etc proc sys dev usr/bin usr/sbin
cp -a ~/idp/busybox/_install/* .
# (Criação dos nós de hardware e compressão)
find . -print0 | cpio --null -ov --format=newc | gzip -9 >
../initramfs.cpio.gz
```

4. Emulação e Resultados

Rodamos o sistema no QEMU:

Shell

```
qemu-system-x86_64 -kernel linux-stable/arch/x86_64/boot/bzImage  
-initrd initramfs.cpio.gz -nographic -append "console=ttyS0  
loglevel=3"
```

Boot: O sistema subiu em **2.51 segundos**.

Token: Ao executar *free*, o Kernel rodou nosso patch. Filtrando com *dmesg*, resgatamos o token: *[+ SOLUCAO: C1B9344556677C4]*

5. Conclusão

Projeto concluído com sucesso. A injeção de código funcionou perfeitamente e o ambiente está estável. Token gerado: **C1B9344556677C4**.